INFORMATION SEARCH AND ANALYSIS SKILLS

(ISAS)

"Understanding Threads and Their Roles in OS"

Group 1

Names: Mufid Fata Rifki

M. Akmal Malik

Dafa Dwi Putra

Class: 1CC6

CEP CCIT

FAKULTAS TEKNIK UNIVERSITAS INDONESIA

2024

PREFACE

First of all, thanks to Allah SWT, who has given us the opportunity to complete this task. Blessing and salutation may also be upon the most honorable prophet Muhammad SAW who has given us life guidance from our birth to the end of life

The title of this paper is “Understanding Threads and Their Roles in OS” and submit it as ISAS task requirements. Our purpose in creating this report is to get to know the thread process, especially the Windows thread. made the entire report carefully and through full analysis

Depok, 11th November 2024

Authors

TABLE OF CONTENTS

PREFACE ..... *i*

TABLE OF CONTENTS ..... *ii*

TABLE OF FIGURES ..... *iv*

CHAPTER I INTRODUCTION *1*

I.1 Background **1**

I.2 Writing Objectives **1**

I.3 Problem Domain **1**

I.4 Writing Methodology **1**

I.5 Writing Framework **2**

CHAPTER II BASIC THEORY *3*

II.1 Operating System **3**

Operating system types **4**

Operating system example **4**

II.2 Introduction to Thread **5**

II.3 Introduction to Process Management **5**

CHAPTER III PROBLEM ANALYSIS *6*

III.1 Thread **6**

Definition **6**

Why do OS need Thread **7**

Thread Components **7**

Types of Thread in Operating System **8**

Difference Between Process and Thread **10**

Multi Threading **11**

Benefits of Thread in Operating System **12**

III.2 Process Management **12**

Characteristics of a process **13**

Process Operations **14**

Killing the Process **15**

Process Scheduling Algorithms **15**

Advantages and Disadvantages **17**

CHAPTER IV CONCLUSION AND SUGGESTION *18*

IV.1 Conclusion **18**

IV.2 Suggestion **18**

BIBLIOGRAPHY ***19***

TABLE OF FIGURES

Figure 3.1.1 Threads example……6

Figure 3.1.2 Thread Component……8

Figure 3.1.3 Disadvantages of User-level threads…….9

Figure 3.1.4 Kernel level thread…….9

Figure 3.1.5 Thread and process differences……10

Figure 3.2.1 Process on memory……13

Figure 3.2.2 Process Operations……14

Figure 3.2.3 Scheduling algorithms……...16

CHAPTER I

INTRODUCTION

I.1 Background

Threads are similar to the gears that keep a computer process turning smoothly. Consider them as small jobs that can function on their own while yet supporting a program's overall efforts. Programs can employ many threads to divide complicated tasks into smaller, more manageable components that operate side by side rather than executing one large activity at a time. Because activities may be completed concurrently rather than in a queue, this technique known as multithreading speeds up and improves the efficiency of systems. Threads enable several actions to occur in sync, which keeps things going smoothly whether you're loading a movie, managing a download, or playing a game.

I.2 Writing Objectives

This report was created with the intention of offering information and understanding of threads generally and specifically in relation to the Windows operating system.

I.3 Problem Domain

The issues the author discussed in this ISAS are as follows:

1. Definition of OS
2. Definition of threads
3. Definition of process

I.4 Writing Methodology

Our writing method is descriptive analysis. Data research techniques such as descriptive analysis help in the description, illustration, or summarization of information points, allowing the development of patterns that fulfill all the requirements of the data. through online resources and user feedback

I.5 Writing Framework

The writing framework for this paper will consist of four chapters, starting with the introduction of this research paper. And then we'll dive into the basic theory, and then we'll explain about further details of the subject. And then finishing off with a conclusion and suggestion.

Chapter I: Introduction

This chapter contains background, writing objectives, problem domain, writing methodology, and writing framework

Chapter II: Basic Theory

This chapter will explain more about the general things about threads, OS, and related things

Chapter III: Problem Analysis

In this chapter, the explanation of threads will be more in-depth by presenting all the details

Chapter IV: Conclusion and Suggestion

This chapter contains conclusions and suggestions regarding the research we are conducting and observe

CHAPTER II

BASIC THEORY

II.1 Operating System

An operating system is an application of software that programs, controls, and facilitates communication between users and hardware by managing hardware and other fundamental system functions.

The main task an operating system carries out is the allocation of resources and services, such as the allocation of memory, devices, processors, and information. The operating system also includes programs to manage these resources, such as a traffic controller, a scheduler, a memory management module, I/O programs, and a file system. The operating system simply provides an environment within which other programs can do useful work.

The operating system enables the control of computer programs through the management of processing time, error checking, input and output control, computing, complicating, storing, and processing data, as well as the execution of various software forms that regulate the execution of related services. The operating system is often the initial software layer that loads into your computer's memory upon startup. The operating system performs the primary functions of the software, while other programs execute after it.

Operating systems are frequently utilized as a channel for information between the user and the hardware of the computers. An operating system handles responsibility as an intermediary between the end-user and the system hardware. Examples include managing computer resources, offering services, and carrying out user programs; presenting an interface to the user; concealing the intricacy of software; enabling several modes of execution; and keeping an eye on user program execution to avoid errors.[1]

Operating system types

There are several ways to categorize operating systems, including how many users they support, how many tasks they can handle at once, what kind of interaction they permit with the system, and the kind of environment they operate in.

1. **Based on Number of Users**: Operating systems can be Single-User or Multi-User. A Single-User OS allows only one user to work on a machine at a time, while a Multi-User OS allows multiple users to work on a device simultaneously.
2. **Based on Number of Tasks**: Operating systems can be Single-Tasking or Multi-Tasking. A Single-Tasking OS can manage only one task at a time, while a Multitasking OS can handle multiple tasks at once.
3. **Based on Interaction**: Operating systems can be characterized as Command-Line Interface (CLI) or Graphical User Interface (GUI). A CLI OS requires commands to be typed, whereas a GUI OS allows users to interact with the system using visual indicators like icons.
4. **Based on Environment**: Operating systems can be categorized as Real-Time, Distributed, Network, Mobile, or Embedded systems. Real-Time OS responds in real-time and is used in embedded systems. Distributed OS uses multiple central processors. Network OS controls and coordinates networked computers. Mobile OS is designed for mobile devices, and Embedded OS is tailored for devices like digital watches and MP3 players.[2]

Operating system example

1. Single User: Microsoft Windows, macOS
2. Multi-User: UNIX, Linux
3. Single-Tasking: MS-DOS
4. Multi-Tasking: Microsoft Windows, macOS, Linux
5. CLI: MS-DOS, Linux (shell)
6. GUI: Microsoft Windows, macOS
7. Real-Time: VxWorks, RTLinux

8. Distributed: Amoeba, Plan9
9. Network: Microsoft Windows Server, Novell NetWare[2]

II.2 Introduction to Thread

A thread is a single sequence stream within a process. Threads are also called lightweight processes as they possess some of the properties of processes. Each thread belongs to exactly one process. In an operating system that supports multithreading, the process can consist of many threads. But threads can be effective only if the CPU is more than 1 otherwise, two threads have to context switch for that single CPU.[3]

II.3 Introduction to Process Management

Process management is a key part of an operating system. By managing the running processes, it regulates how processes are executed and how your computer functions. Among many other things, this entails halting processes and determining which ones require greater attention. On your personal computer, you can also manage processes.[4]

CHAPTER III

PROBLEM ANALYSIS

III.1 Thread

Definition

A thread is a single sequential flow of execution of tasks of a process so it is also known as thread of execution or thread of control. There is a way of thread execution inside the process of any operating system. Apart from this, there can be more than one thread inside a process. Each thread of the same process makes use of a separate program counter and a stack of activation records and control blocks. Thread is often referred to as a lightweight process.

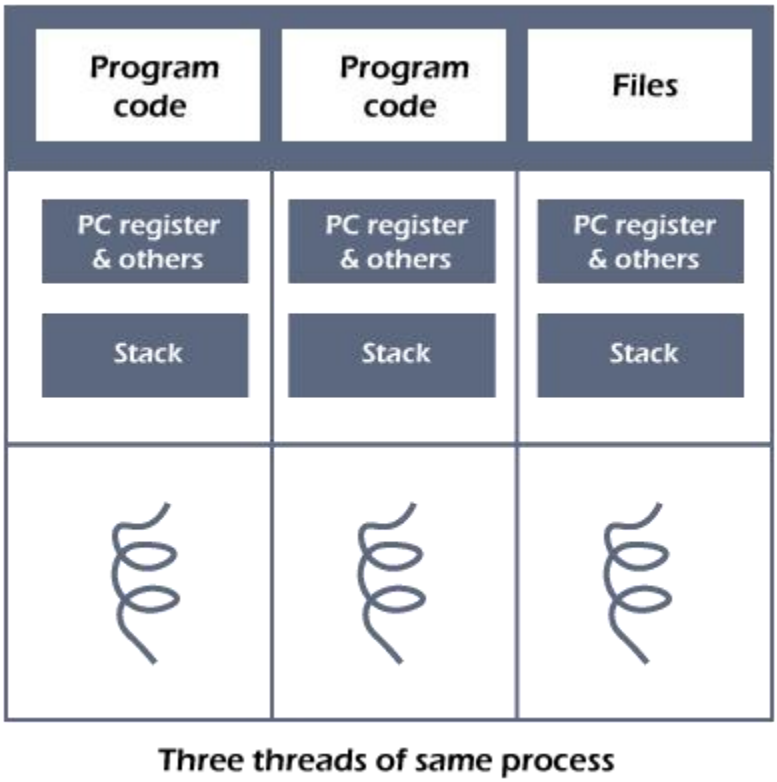

Figure 3.1.1 Threads example

(Ref: https://images.javatpoint.com/operating-system/images/threads-in-operating-system.png)

The process can be split down into so many threads. For example, in a browser, many tabs can be viewed as threads. MS Word uses many threads - formatting text from one thread, processing input from another thread, etc.[5]

Why do OS need Thread

1. The performance of the application is enhanced by threads operating in parallel. These threads share the process's address space and environment, but each has its own CPU state and stack.

2. Inter-process communication is not necessary because threads can share common data. Threads have states like ready, executing, blocked, etc., just like processes do.

3. Similar to the process, the threads can be given priorities, with the highest priority thread being scheduled first.

4. Every thread has a Thread Control Block (TCB) of its own. Register contents are saved in (TCB), and the thread experiences a context switch similar to that of the process. Because threads share resources and address space, synchronization is also necessary for the thread's numerous operations.[3]

Thread Components

These are the basic component of Thread

1. **Stack space**

a section of memory allocated to a thread or process that is used to hold temporary information, including return addresses, local variables, and function arguments.

2. **Register set**

Registers are used to store values like program counters, instruction pointers, and data being processed.

3. **Program counter**

a register that holds the memory location of the subsequent command that the CPU will execute.

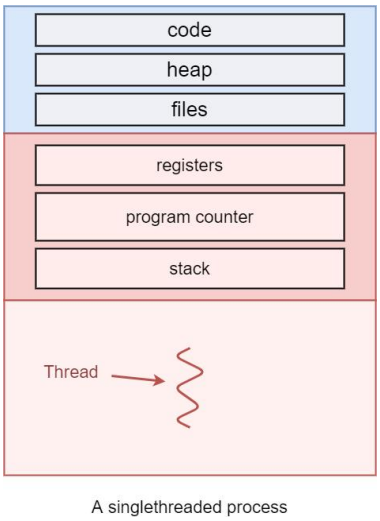

Figure 3.1.2 Thread Component

(Ref:https://kuleuven-diepenbeek.github.io/osc-course/img/os/db_singlethreaded_v2.png)

Types of Thread in Operating System

There are two kinds of threads.

1. **User Level Thread**

The user-level thread is not recognized by the operating system. User threads are simple to implement and are done by the user. The entire process is blocked if a user executes a user-level thread blocking operation. The user-level thread is completely unknown to the kernel level thread.Let's take a look at the advantages and disadvantages of User-Level Thread.

Advantages:

1. The user threads can be more easily implemented than the kernel thread.
2. User-level threads can be applied to such types of operating system that do not support threads at the kernel-level.
3. It does not require modifications of the operating system.

Disadvantages:

1. User-level threads lack coordination between the threads and the kernel.
2. If a thread causes a page fault, the entire process is blocked.

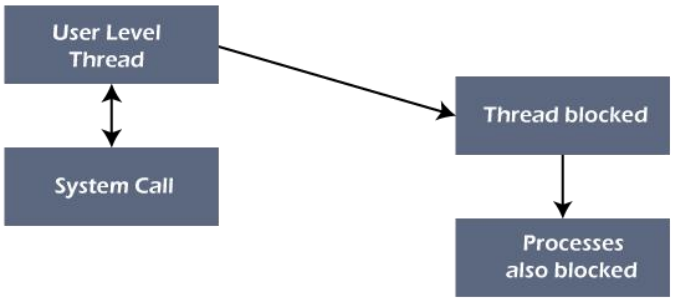

Figure 3.1.3

(Ref:https://images.javatpoint.com/operating-system/images/threads-in-operating-system2.png)

2. Kernel Level Thread

The operating system is recognized by the kernel thread. For every kernel-level thread and process, the system has a thread control block and a process control block. The operating system implements the kernel-level thread. The kernel controls and is aware of every thread. To construct and control the threads from user space, the kernel-level thread provides a system function. Compared to user threads. The kernel thread has a longer context switch time. The banky thread can continue to execute if a blocking operation is carried out by a kernel thread. Let's take a look at the advantages and disadvantages of User-Level Thread.

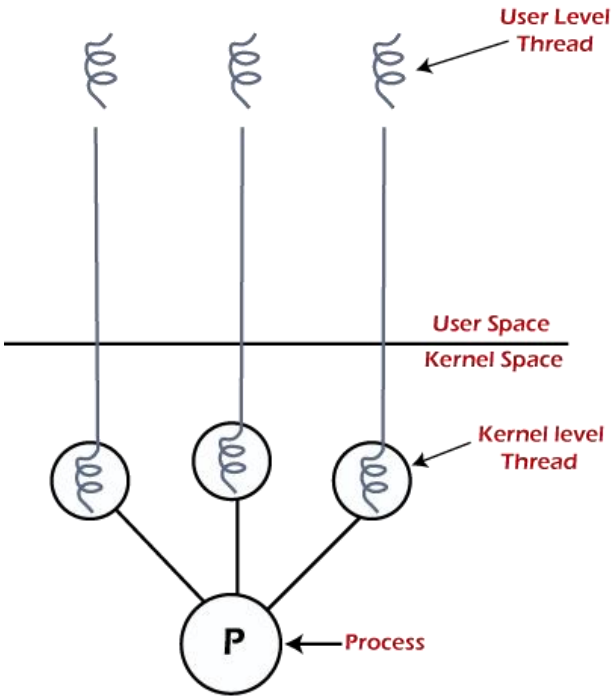

Figure 3.1.4 Kernel level thread

(Ref: https://images.javatpoint.com/operating-system/images/threads-in-operating-system3.png)

Advantages:

1. The kernel-level thread is fully aware of all threads.
2. The scheduler may decide to spend more CPU time in the process of threads being large numerical.
3. The kernel-level thread is good for those applications that block the frequency.

Disadvantages:

1. The kernel thread manages and schedules all threads.
2. The implementation of kernel threads is difficult than the user thread
3. The kernel-level thread is slower than user-level threads.[5]

Difference Between Process and Thread

The main difference is that threads within the same process run in a shared memory space, whereas processes run in separate memory spaces. Threads are not independent of each other like processes are. As a result, threads share their code section, data section and operating system resources (such as open files and signals) with other threads. However, like a process, a thread has its own programme counter (PC), register set and stack space.

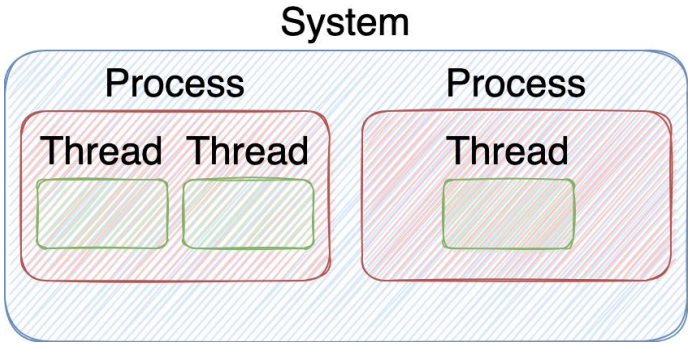

Figure 3.1.5 Thread and Process difference

(Ref:https://miro.medium.com/v2/resize:fit:1400/1*rjRfIdFwHC4i5UzkzVvOWw.png)

Multi Threading

Another name for a thread is a lightweight process. The goal is to divide a process into several threads in order to achieve parallelism. For instance, several tabs in a browser may represent several threads. Multiple threads are used by Microsoft Word, including one for text formatting and another for input processing.

Multi threading is a technique used in operating systems to improve the performance and responsiveness of computer systems. Multi threading allows multiple threads (i.e., lightweight processes) to share the same resources of a single process, such as the CPU, memory, and I/O devices.

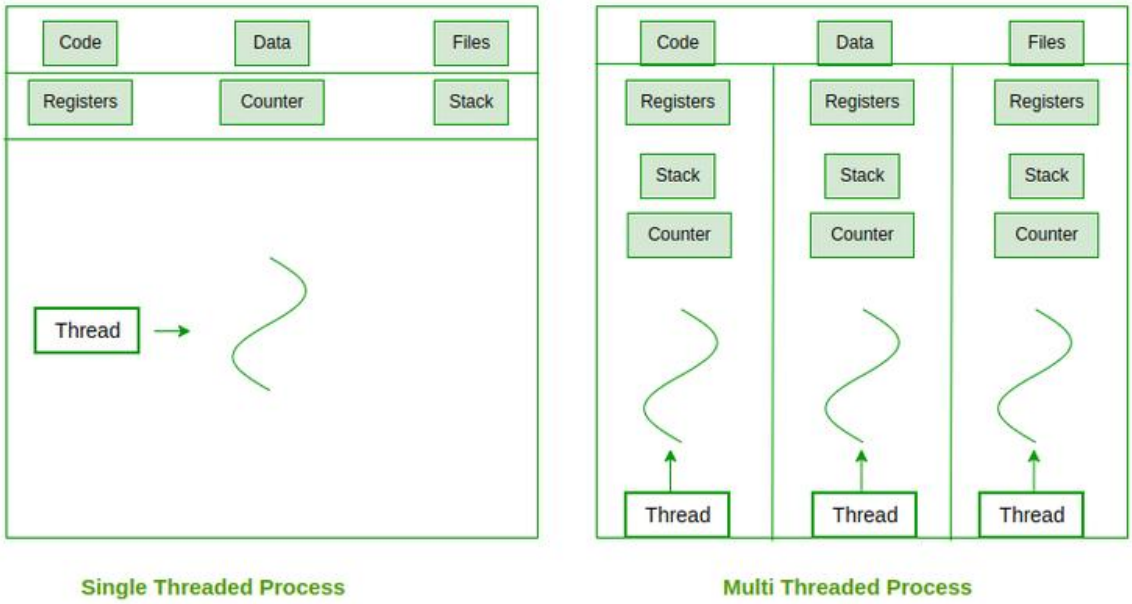

Figure 3.1.6 Multi thread process

(Ref: https://media.geeksforgeeks.org/wp-content/uploads/20240226114918/Screenshot-from-2024-02-26-11-48-56-768.png

Benefits of Thread in Operating System

Here are the benefits of threads:

1. Enhanced throughput of the system: The number of jobs completed in a given amount of time rises when the process is divided into numerous threads, each of which is handled as a job. For this reason, the system's throughput as well rises.
2. Effective Utilization of Multiprocessor system: You can schedule multiple threads in multiple processors when you have many threads in a single process.
3. Faster context switch: The context switching period between threads is less than the process context switching. The process context switch means more overhead for the CPU.
4. Responsiveness: When a process is divided into multiple threads, it can be responded to as quickly as possible after each thread has finished running.
5. Communication: While we only use a few exclusive communication mechanisms for communication between two processes in a process, multiple-thread communication is straightforward since the threads share the same address space.
6. Resource sharing: All threads in a process can share resources, including files, data, and code. Note: Threads are unable to share the register or stack. Every thread has its own register and stack.[5]

III.2 Process Management

Process management is one of the most vital parts of any operating system. It regulates the operation and running of processes, thereby regulating how your computer works. This entails stopping the processes, among many other things, and making it decide which ones need more attention. You can do this on your personal computer, too.

OS manages the creation, termination, and scheduling of processes. A process is a program in execution. Several synchronization methods are used in OS to synchronize deadlock-free processes and inter-process communication. Good process management ensures maximum system throughput without any clash in process execution and proper allocation of resources. This essential element of the operating system enhances the system utilization by permitting the execution of more than one application concurrently. Below is what is process look like on memory:

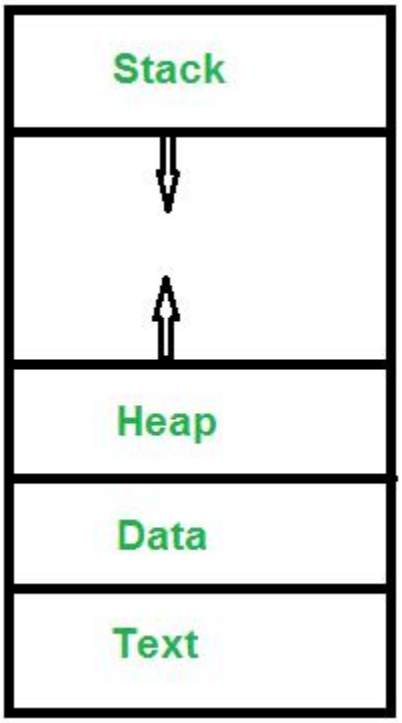

Figure 3.2.1 Process on memory

(Ref: https://www.geeksforgeeks.org/introduction-of-process-management/?ref=header_outind)

1. Text Section: The value of the Program Counter indicates the current activity, which is also included in a Process, also referred to as the Text Section.
2. Stack: The stack holds temporary data, including function parameters, return addresses, and local variables.
3. Data Section: Contains the global variable.
4. Heap Section: Dynamically allocated memory to process during its run time.[6]

Characteristics of a process

1. **Process Id**: A unique identifier assigned by the operating system.
2. **Process State**: Can be ready, running, etc.
3. **CPU Registers**: Like the Program Counter (CPU registers must be saved and restored when a process is swapped in and out of the CPU)
4. **Accounts Information**: Amount of CPU used for process execution, time limits, execution ID, etc
5. **I/O Status Information**: For example, devices allocated to the process, open files, etc

6. **CPU Scheduling Information**: For example, Priority (Different processes may have different priorities, for example, a shorter process assigned high priority in the shortest job first scheduling)

Process Operations

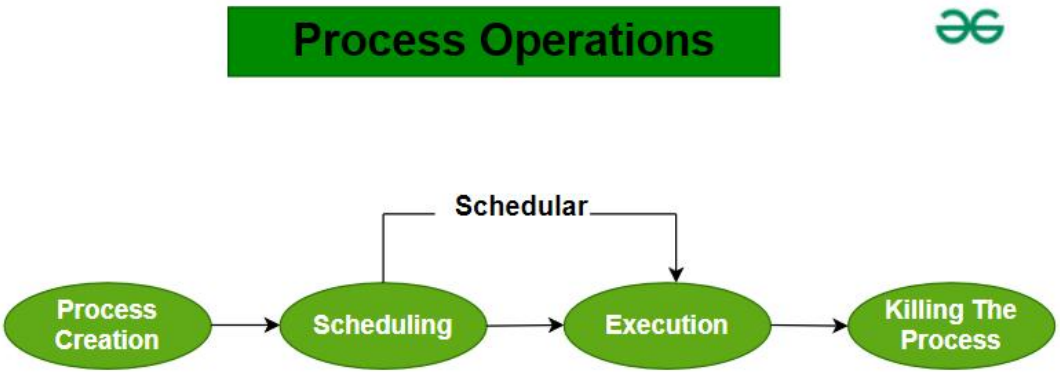

Figure 3.2.2 Process Operations
(Ref:https://media.geeksforgeeks.org/wp-content/uploads/20240709113806/Screenshot-2024-07-09-113355.png)

Process operations in an operating system refer to the various activities the OS performs to manage processes. These operations include process creation, process scheduling, execution and killing the process. Here are the key process operations:

Process Creation

Process creation in an operating system (OS) is the act of generating a new process. This new process is an instance of a program that can execute independently.

Scheduling

Once a process is ready to run, it enters the “ready queue.” The scheduler’s job is to pick a process from this queue and start its execution.

Execution

Execution means the CPU starts working on the process. During this time, the process might:

1. Move to a waiting queue if it needs to perform an I/O operation.
2. Get blocked if a higher-priority process needs the CPU.

Killing the Process

After the process finishes its tasks, the operating system ends it and removes its Process Control Block (PCB).

Process Scheduling Algorithms

In operating systems, scheduling is the method by which processes are given access to system resources, primarily the CPU. Efficient scheduling is essential for optimal system performance and user satisfaction. There are two primary types of CPU scheduling: preemptive and non-preemptive.

When a process moves from the running state to the ready state or from the waiting state to the ready state, preemptive scheduling is employed. The process is given a set length of time to use the resources (mostly CPU cycles), after which they are removed. If there is still CPU burst time left, the process is then put back in the ready queue. Until it has another opportunity to run, that process remains in the ready queue.

Non-preemptive Scheduling is used when a process terminates, or a process switches from running to the waiting state. In this scheduling, once the resources (CPU cycles) are allocated to a process, the process holds the CPU till it gets terminated or reaches a waiting state. In the case of non-preemptive scheduling does not interrupt a process running CPU in the middle of the execution. Instead, it waits till the process completes its CPU burst time, and then it can allocate the CPU to another process.[7]

The operating system can use different scheduling algorithms to schedule processes. Here are some commonly used timing algorithms:

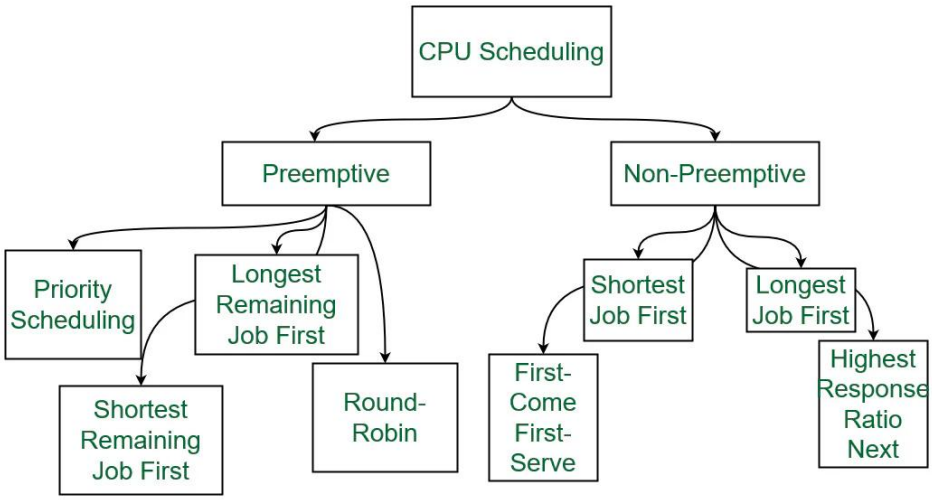

Figure 3.2.3 Scheduling algorithms

(Ref: https://cdn.shopify.com/s/files/1/0329/9865/3996/t/5/assets/cpu_scheduling_algorithms_in_os-pOEBo2.True?v=1707751006)

1. **First-Come, First-Serve (FCFS)**: The most straightforward CPU scheduling mechanism is First-Come, First-Served (FCFS). As the name implies, it carries out operations in the order that they are received, starting with the first process to run.
2. **Shortest Job Next (SJN)**: The process with the least total execution time is chosen next by the Shortest Job Next (SJN) scheduling algorithm, often referred to as Shortest Job First (SJF). Since it is a non-preemptive algorithm, no process can stop another process from running once it has begun.
3. **Round Robin (RR)**: One of the CPU scheduling techniques is called Round-Robin (RR), which causes all of its tasks to run cyclically. To put it another way, this algorithm will execute a process within a predetermined amount of time, and if it has exceeded that period, it will automatically pause and move to the very back of the queue, where it will then proceed to execute another process from the front queue.
4. **Priority Scheduling**: The Priority Scheduling Algorithm assigns a priority value to each process based on factors such as priority level, memory requirements, or other specified criteria. The process with the highest priority is executed first, followed by processes with lower priorities.
5. **Multi-level Queue Scheduling:** Depending on their priority, processes are grouped into several queues, each of which has an own scheduling algorithm.[8]

6. **Highest Response Ratio Next (HRRN):** The non-preemptive technique known as Highest Response Ratio Next (HRNN) bases scheduling on an additional parameter known as Response Ratio. Each of the open positions has its response ratio determined, and the job with the highest response ratio is prioritized above the others.[9]
7. **Longest Job First (LJF):** The scheduling algorithm known as Longest Job First (LJF) is non-preemptive. The burst time of the processes is the foundation of this algorithm. Based on their burst times, or in descending order of burst times, the processes are added to the ready queue. This algorithm is predicated on the idea that the process with the longest burst duration is handled first, as the name implies.[10]
8. **Longest Remaining Time First(LRTF):** An preemptive variant of the Longest Job First (LJF) scheduling technique is called Longest Remaining Time First (LRTF). In this scheduling technique, we discover the process with the maximum remaining time and then process it, i.e. check for the maximum remaining time after some interval of time(say 1 unit each) to check if another process having higher Burst Time arrived up to that moment.[11]
9. **Shortest Remaining Time First:** The process with the least amount of time left to complete is chosen to run in the Shortest Remaining Time First (SRTF) scheduling method. Processes will always run until they are finished or a new process that needs less time is added since the process that is presently running is the one with the least amount of time left by definition, and that time should only decrease as execution continues.[12]

Advantages and Disadvantages

Advantages:

1. **Running Multiple Programs**: Process management lets you run multiple applications at the same time, for example, listening to music while browsing the web.
2. **Process Isolation**: It ensures that different programs don't interfere with each other, so a problem in one program won't crash another.
3. **Fair Resource Use**: It makes sure resources like CPU time and memory are shared fairly among programs, so even lower-priority programs get a chance to run.
4. **Smooth Switching**: It efficiently handles switching between programs, saving and loading their states quickly to keep the system responsive and minimize delays.

Disadvantages:

Overhead: Because the OS must monitor many data structures and scheduling queues, process management consumes system resources. Memory and CPU time are needed for this, which may impact system performance.

Complexity: The requirement for precise scheduling algorithms and resource allocation techniques makes OS design and maintenance challenging.

Deadlocks: To keep processes running smoothly together, the OS uses mechanisms like semaphores and mutex locks. However, these can lead to deadlocks, where processes get stuck waiting for each other indefinitely.

Increased Context Switching: In multitasking systems, the OS frequently switches between processes. Storing and loading the state of each process (context switching) takes time and computing power, which can slow down the system.

CHAPTER IV

CONCLUSION AND SUGGESTION

IV.1 Conclusion

Lightweight processes inside a single process, known as threads, provide a potent way to improve application performance. Threads facilitate effective communication and resource usage by sharing the same address space and resources. Faster context switching, better resource sharing, and faster system response times are all made possible by this concurrency.

Both kernel-level and user-level threads, each with unique advantages and disadvantages, add to multithreading's overall efficacy. This method is fundamental to effective application development and is necessary for contemporary computer systems since it greatly increases speed and responsiveness.

IV.2 Suggestion

In contemporary operating systems, threads also known as lightweight processes are essential because they allow for effective multitasking and concurrency within a single process. The significance of threads in improving system performance, responsiveness, and resource utilization is examined in this research paper, which explores the intricate relationship between threads and the operating system.

The purpose of this study is to clarify the opportunities and difficulties related to thread-based programming by investigating the different thread models, scheduling methods, and synchronization mechanisms. Additionally, it looks into how threads affect operating system design, including implementations at the kernel and user levels, as well as how they affect security and fault tolerance.

BIBLIOGRAPHY

[1] Function of operating system GeeksforGeeks. (2024, July 1). Functions of operating systems. GeeksforGeeks. https://www.geeksforgeeks.org/functions-of-operating-system/

[2] *Operating Systems: Functions, Types, Examples | StudySmarter*. (n.d.). StudySmarter UK. https://www.studysmarter.co.uk/explanations/computer-science/computer-systems/operating-systems

[3] GeeksforGeeks. (2024, June 27). *Thread in operating system*. GeeksforGeeks. https://www.geeksforgeeks.org/thread-in-operating-system/

[4] GeeksforGeeks. (2024, July 12). Introduction of Process Management. GeeksforGeeks. https://www.geeksforgeeks.org/introduction-of-process-management/

[5] *Threads in Operating System (OS) - javatpoint*. (n.d.). www.javatpoint.com. https://www.javatpoint.com/threads-in-operating-system

[6] GeeksforGeeks. (2024b, July 12). *Introduction of Process Management*. GeeksforGeeks. https://www.geeksforgeeks.org/introduction-of-process-management/

[7] GeeksforGeeks. (2024, September 26). *Preemptive and NonPreemptive Scheduling*. GeeksforGeeks. https://www.geeksforgeeks.org/preemptive-and-non-preemptive-scheduling/

[8] Chebitko, R. (2024, February 13). *CPU Scheduling Algorithms In OS*. Softwareg.com.au. https://softwareg.com.au/blogs/computer-hardware/cpu-scheduling-algorithms-in-os

[9] *OS HRRN Scheduling - javatpoint*. (n.d.). www.javatpoint.com. https://www.javatpoint.com/os-hrrn-scheduling

[10] GeeksforGeeks. (2023, February 13). *Longest Job First (LJF) CPU Scheduling Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/longest-job-first-ljf-cpu-scheduling-algorithm/

[11] GeeksforGeeks. (2024a, March 12). *Longest Remaining Time First (LRTF) or Preemptive Longest Job First CPU Scheduling Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/longest-remaining-time-first-lrtf-cpu-scheduling-algorithm/

[12] GeeksforGeeks. (2024b, May 7). *Shortest Remaining Time First (Preemptive SJF) Scheduling Algorithm*. GeeksforGeeks. https://www.geeksforgeeks.org/shortest-remaining-time-first-preemptive-sjf-scheduling-algorithm/